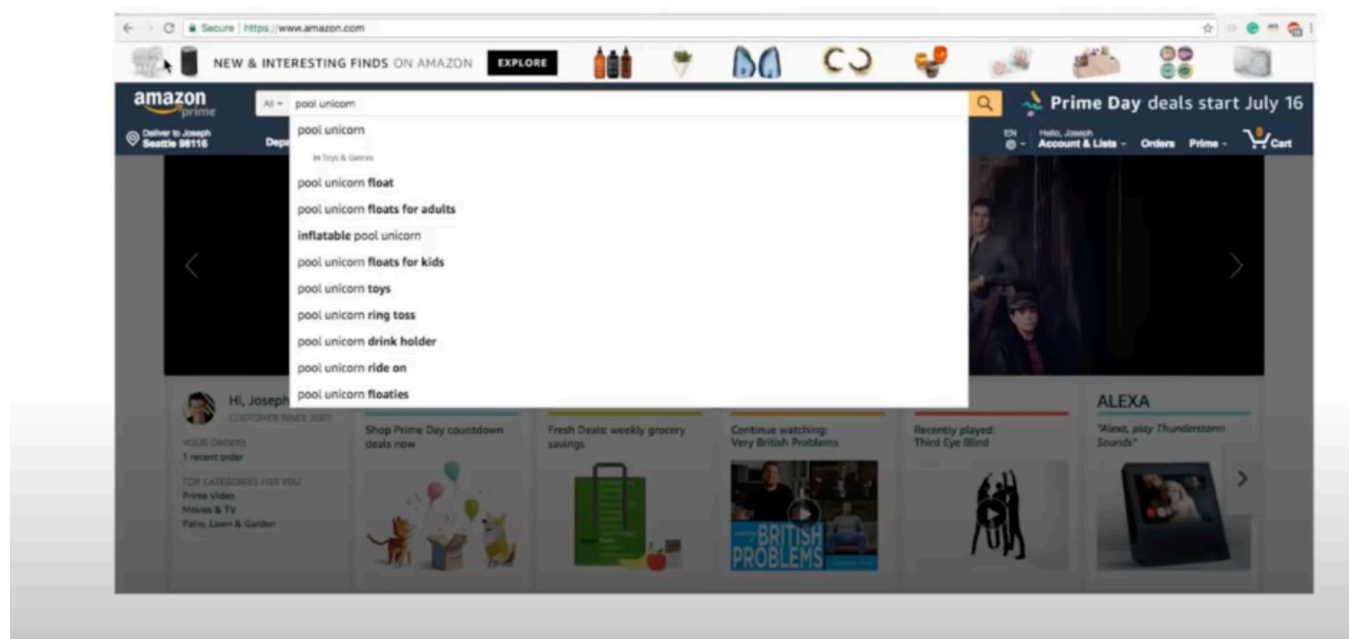


Going Beyond SQL's "LIKE" for Lightning-Fast Results

🌐 systemdesignschool.io/fundamentals/full-text-search-database



Full-text Search Database

Document databases store JSON efficiently. But searching text within documents requires different technology. Product descriptions, blog posts, and customer reviews need full-text search.

The LIKE Keyword Problem

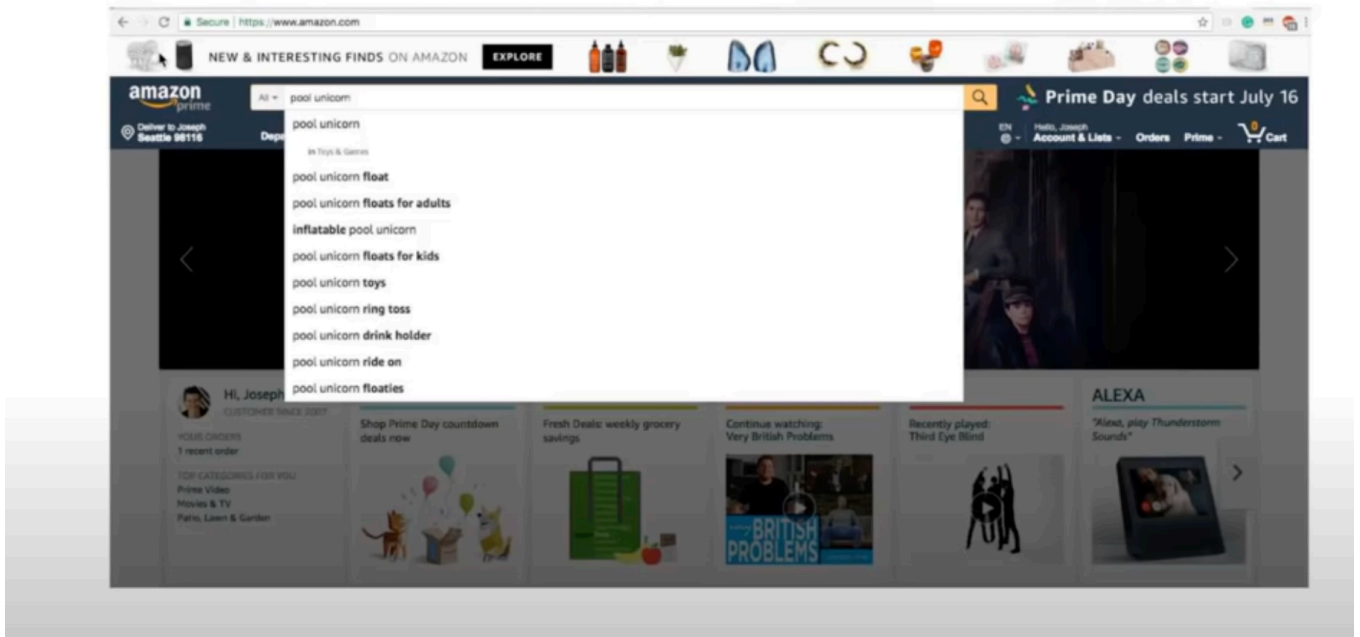
SQL databases provide the **LIKE** keyword for text search. Search for products containing "wireless headphones":

```
SELECT * FROM products WHERE description LIKE '%wireless%headphones%';
```

SQL scans every row. Check the description column. Does it match the pattern? Include it in results. This full table scan examines millions of rows. A products table with 10 million entries takes seconds to search.

The wildcards make indexing impossible. B-tree indexes work for prefix matches. **LIKE** 'apple%' uses an index. But **LIKE** '%apple%' cannot. The database must read every row.

Large tables make this impractical. E-commerce sites need sub-second search. SQL **LIKE** can't deliver. Full-text search databases solve this problem.



Inverted Index

Inverted indexes map terms to documents. Unlike a book's table of contents mapping pages to topics, an inverted index maps topics to pages. This reversal enables fast lookups.

Consider a collection of documents with their respective document IDs:

Document ID	Document
1	The cat is black
2	The dog is brown
3	The mouse is grey
4	The cat and the dog are friends

To create an inverted index, we first tokenize the documents by splitting them into individual terms and then associate each term with the corresponding document IDs where it appears. Here's how the inverted index would look for the given example:

Term	Document IDs
and	4
are	4
black	1
brown	2

Term	Document IDs
cat	1, 4
dog	2, 4
friends	4
grey	3
is	1, 2, 3
mouse	3
the	1, 2, 3, 4

Find documents containing "cat"? Look up "cat" in the index. Return documents 1 and 4. No table scan needed. The lookup takes constant time.

The index can store additional metadata. Term frequencies enable ranking. Term positions support phrase queries. Timestamps enable recency sorting. This metadata powers advanced search features.

Popular Full-Text Search Databases

There are several full-text search databases available today, each with its unique features and optimizations. Some popular choices include:

Database Name	Cloud Provider	Distinct Feature
Elasticsearch	AWS, GCP, Azure, Open Source	Probably the most popular full-text search database out there. Real-time analytics, extensive query capabilities, and horizontal scaling for large data volumes.
Amazon CloudSearch	AWS	Seamless integration with other AWS services, automatic scaling, and language-specific text processing.
Azure Cognitive Search	Azure	AI-powered features like semantic search, computer vision, and natural language processing capabilities.
Google Cloud Search	GCP	Machine learning-based search quality improvement and integration with G Suite applications.
Algolia	Other (SaaS)	Comprehensive typo-tolerance, and customizable ranking rules.
Apache Solr	Open Source	Advanced faceted search, distributed searching, and extensible plugin architecture.
MeiliSearch	Open Source	Low memory footprint, instant search-as-you-type experience, and straightforward API integration.

Demo

Here is an example of a more realistic e-commerce application using Elasticsearch and Python.

We would first create an Elasticsearch index with a custom mapping that defines the fields and their types for a product document. We would also have more fields, such as price, category, and description, and handle features like pagination, sorting, and filtering.

```
from elasticsearch import Elasticsearch # Initialize Elasticsearch client es =
Elasticsearch("http://localhost:9200") # Define the index mapping for products
mapping = { "mappings": { "properties": { "name": {"type": "text"},
"description": {"type": "text"}, "price": {"type": "double"}, "category":
{"type": "keyword"}, } } } # Create the index with the mapping
es.indices.create(index="products", body=mapping) # Index sample products
products = [ {"id": 1, "name": "Apple iPhone 13", "description": "Latest iPhone
model with a 120Hz display.", "price": 999.0, "category": "smartphones"}, {"id":
2, "name": "Samsung Galaxy S22", "description": "Flagship Samsung smartphone
with a versatile camera system.", "price": 899.0, "category": "smartphones"},
{"id": 3, "name": "Google Pixel 6", "description": "Google's newest smartphone
with stock Android experience.", "price": 799.0, "category": "smartphones"},
{"id": 4, "name": "Apple MacBook Pro", "description": "High-performance laptop
with Apple's M1 Pro chip.", "price": 2499.0, "category": "laptops"}, ] for
product in products: es.index(index="products", id=product["id"], body=product)
# Refresh the index to make the newly added documents searchable
es.indices.refresh(index="products") # Simulate user input user_input =
"smartphone" # Define the search query with pagination, sorting, and filtering
query = { "from": 0, # Starting index for pagination "size": 10, # Number of
results per page "query": { "multi_match": { "query": user_input, "fields":
["name", "description"], } }, "sort": [ {"price": {"order": "asc"}} ], "filter":
{ "term": {"category": "smartphones"} } } response = es.search(index="products",
body=query) # Print search results for hit in response["hits"]["hits"]:
print(hit["_source"])
```

If you would like to try it on your local computer, install elasticsearch on your local system and the python package using `pip install elasticsearch`.

If you would like to see a more complete demo, check out ElasticSearch's official [movie demo](#).

Use Cases

Any search box likely uses full-text search. Several common patterns appear across industries.

E-commerce platforms like Amazon and eBay rely on full-text search. Users search for products using keywords. The search engine ranks results by relevance, price, and reviews. Typo tolerance helps users find "wireles headphones" even with misspellings.

Social media platforms including Twitter and Facebook enable content discovery. Users search for posts, profiles, and hashtags. Results rank by recency, relevance, and engagement. Real-time indexing shows new content immediately.

Document management systems like Dropbox and Google Drive search file names and contents. Users find documents by searching text within PDFs and documents. Metadata filters narrow results by file type and modification date.

Forums and Q&A platforms such as Stack Overflow and Reddit help users find discussions. Search keywords, phrases, or tags. Results rank by recency, relevance, and community votes. Answers to common questions surface first.

For deeper coverage of search engine internals, see the [Search Engine Mechanics](#) section. It covers [inverted index fundamentals](#), [TF-IDF scoring](#), [BM25 ranking](#), and [Elasticsearch architecture](#).